

Edge Deployment of Signal Models: INT8 QAT vs PTQ

Which models quantize and which collapse, measured

2i

2026-07-29

Abstract

Places that actually want to run signal models on the edge usually have no GPU. The real deployment targets are cheap on premises CPU servers, embedded boards, and router grade processors, and the budget is far smaller than a cloud GPU inference budget. In that setting, quantizing a model to INT8 is known to shrink both size and latency at once. But the common belief that ‘quantization gives you a free 3x shrink’ often hides an accuracy cost this paper puts a number on. We compared two INT8 quantization paths on a single small automatic modulation classification (AMC) CNN, post training quantization (PTQ, calibration only, no retraining) and quantization aware training (QAT, quantization folded into the training loop with a short fine tune), using the same data, the same model, and the same measurement harness. The result is simple and extreme. Starting from an FP32 baseline accuracy of 93.18 percent, PTQ collapsed to 61.91 percent, a loss of 31.27 percentage points. The same model given 15 epochs of quantization aware fine tuning recovered to 91.45 percent, a loss of only 1.73 percentage points. Both paths cut model size identically, from 119.3 KB to 35.5 KB, a 70.3 percent reduction, and both cut inference latency by more than 43 percent at batch size 1. The size and latency gains are identical between the two paths, only the accuracy cost differs. PTQ pays nearly the entire cost; QAT recovers 94.5 percent of it. The conclusion is direct. Deploying an INT8 signal classification model on cheap on premises edge hardware is not viable with PTQ alone; QAT should be the default path.

Contents

1	Abstract	3
2	Why Edge Quantization Matters	3
3	Two Quantization Paths: PTQ vs QAT	3
4	Data and Experimental Setup	4
5	Results	4
6	Limitations	6
7	Reproduction	6
8	References	7

1 Abstract

Places that actually want to run signal models on the edge usually have no GPU. The real deployment targets are cheap on premises CPU servers, embedded boards, and router grade processors, and the budget is far smaller than a cloud GPU inference budget. In that setting, quantizing a model to INT8 is known to shrink both size and latency at once. But the common belief that “quantization gives you a free 3x shrink” often hides an accuracy cost this paper puts a number on. We compared two INT8 quantization paths on a single small automatic modulation classification (AMC) CNN, post training quantization (PTQ, calibration only, no retraining) and quantization aware training (QAT, quantization folded into the training loop with a short fine tune), using the same data, the same model, and the same measurement harness. The result is simple and extreme. Starting from an FP32 baseline accuracy of 93.18 percent, PTQ collapsed to 61.91 percent, a loss of 31.27 percentage points. The same model given 15 epochs of quantization aware fine tuning recovered to 91.45 percent, a loss of only 1.73 percentage points. Both paths cut model size identically, from 119.3 KB to 35.5 KB, a 70.3 percent reduction, and both cut inference latency by more than 43 percent at batch size 1. The size and latency gains are identical between the two paths, only the accuracy cost differs. PTQ pays nearly the entire cost; QAT recovers 94.5 percent of it. The conclusion is direct. Deploying an INT8 signal classification model on cheap on premises edge hardware is not viable with PTQ alone; QAT should be the default path.

2 Why Edge Quantization Matters

Running tasks like radio signal classification, interference detection, or device fingerprinting on cloud GPUs is convenient at the research stage, but it is usually not an option at the actual deployment point. Inference modules built into communications equipment often have no power or space budget for a GPU. Security requirements frequently forbid sending data off premises, forcing inference onto local servers, and those servers are cheap CPU boxes or low power processors embedded in a router, not cloud GPU instances. Running a full 32 bit floating point (FP32) model on that hardware means a large memory footprint and long inference latency.

INT8 quantization is the standard answer to this problem. Representing weights and activations as 8 bit integers instead of 32 bit floats theoretically shrinks model size fourfold, and on CPUs that support integer kernels, inference runs faster than floating point. The problem is that this gain is not free. Mapping floating point values onto 8 bit integers reduces representational precision, and how that precision loss affects model accuracy depends heavily on the model and the data. Large language models and image classification CNNs have many reported cases where INT8 PTQ works with little to no accuracy loss, and it is tempting to assume that optimism carries over to signal models without checking. This paper checks that assumption directly.

Signal models differ from image classification CNNs in one important way. The input is IQ (in phase, quadrature phase) time series data, and batch normalization statistics near the front of the network play a critical role in stabilizing the signal scale. The standard PTQ path folds batch normalization into the convolution and then freezes activation scales from calibration data, and this order of operations may be particularly fragile for signal models. That was the core hypothesis behind this experiment, and the sections below test it with measurement, not assumption.

3 Two Quantization Paths: PTQ vs QAT

Both paths converge on the same destination, integer arithmetic kernels, but they take different routes to get there.

PTQ (post training quantization) starts from an already trained FP32 model and (1) fuses convolution, batch normalization, and ReLU into a single fused module, (2) inserts observers and runs a portion of the validation data through the model to calibrate each layer’s activation distribution, and (3) fixes the resulting scale and zero point values and converts the model to integer kernels. There is no retraining at all. It is a cheap path that finishes in seconds once a batch of calibration data has passed through.

QAT (quantization aware training) starts from the same FP32 checkpoint but fine tunes for a few additional epochs with fake quantization operators inserted into the forward pass to simulate quantization during training. In this experiment, batch normalization was not immediately folded and frozen; instead it was kept in a fused module (an intrinsic ConvBnReLU1d module) that continues to update during training. Late in fine tuning, observers were frozen first, followed by freezing the batch normalization running statistics, a standard stabilization order. As measured directly in this experiment (see Section 4), skipping this order leads to non convergent or oscillating training. Once fine tuning finishes, the model is converted to integer kernels exactly as in PTQ.

Both paths ultimately use the same class of integer arithmetic kernel, so the upper bound on size and latency gain is, in principle, identical between them. The only difference is how much accuracy each path preserves. The only extra cost QAT pays is the wall clock time of a few additional fine tuning epochs added to the training pipeline.

4 Data and Experimental Setup

The data is a 6dB signal to noise ratio (SNR) subset of the public wireless signal benchmark RadioML2016.10a. It is an 11 class modulation classification task (8PSK, AM-DSB, AM-SSB, BPSK, CPFSK, GFSK, PAM4, QAM16, QAM64, QPSK, WBFM) over IQ time series input (2 channels, length 128). The split used 8,415 samples for training, 1,485 for validation (early stopping), and 1,100 for testing. The test set was never used in either the FP32 training run or the QAT fine tuning run, so there is no data leakage.

The model is a small CNN made of three Conv1d blocks (each including batch normalization and ReLU), global average pooling, and two fully connected layers. The parameter count is in the tens of thousands, sized for edge deployment. The FP32 baseline trained for 80 epochs (early stopping patience 20) and reached a validation accuracy of 92.996 percent and a test accuracy of 93.18 percent. Training used label smoothing of 0.05.

PTQ applies an eager mode static quantization API on top of this model (fuse_modules to fuse convolution, batch normalization, and ReLU), calibrates observers on the validation set, and converts to integer kernels. The backend is qnnpack, a quantization engine natively optimized for ARM class CPUs including Apple Silicon.

QAT groups convolution, batch normalization, and ReLU into a trainable fused module (fuse_modules_qat), inserts fake quantization observers on weights and activations, and fine tunes for 15 epochs at a learning rate of $3e-5$ (one tenth of the FP32 training learning rate). Observers were frozen starting at epoch 11, and batch normalization running statistics were frozen starting at epoch 13, with training continuing through epoch 15. The model was then converted to integer kernels.

Three quantities were measured. Test accuracy was computed by comparing argmax predictions against labels directly on the test set. Model size was the actual byte count of the serialized state dictionary. Inference latency was measured at batch size 1 on CPU, after 30 warmup runs, over 300 measured forward passes, reporting p50 and p95. Every number reported is a value the measurement code computed directly, not a value the model self reported.

5 Results

The table below compares the FP32 baseline, PTQ, and QAT within the same run, on the same test set, using the same measurement harness.

Configuration	Test Accuracy	Accuracy Delta	Model Size	Size Delta	p50 Latency	p50 Delta
FP32 (baseline)	93.18%	-	119.3 KB	-	0.278 ms	-
INT8 PTQ (calibration only)	61.91%	-31.27pp	35.5 KB	-70.3%	0.158 ms	-43.2%

Configuration	Test Accuracy	Accuracy Delta	Model Size	Size Delta	p50 Latency	p50 Delta
INT8 QAT (15 epoch fine tune)	91.45%	-1.73pp	35.5 KB	-70.3%	0.155 ms	-44.2%

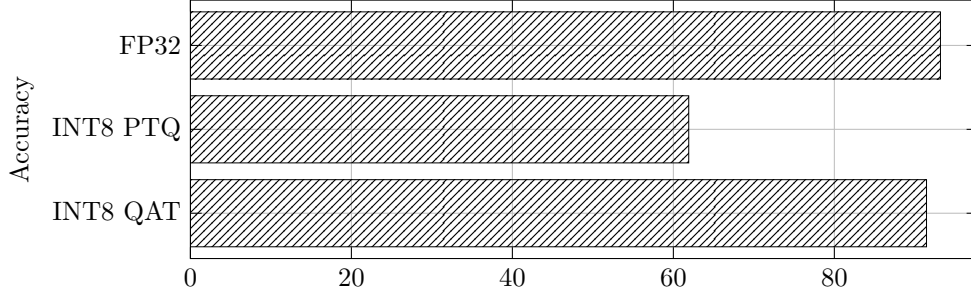


Figure 1: Test Accuracy Comparison

The first finding is that size and latency gains are effectively identical between PTQ and QAT. Both paths convert to the same qnnpack integer kernels, so the fine tuning stage does not change the inference graph itself. QAT’s measured p50 latency, 0.155 ms, was in fact slightly lower than PTQ’s 0.158 ms (within measurement noise, but it at minimum confirms QAT does not sacrifice the latency gain).

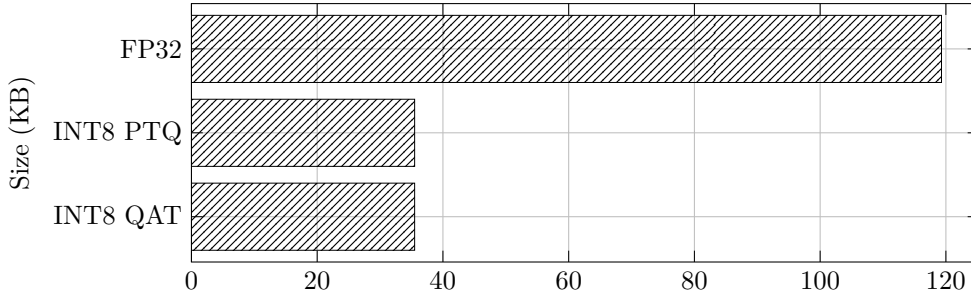


Figure 2: Model Size Comparison

The second, and central, finding is about accuracy. PTQ with calibration alone dropped accuracy from 93.18 percent to 61.91 percent, a 31.27 percentage point loss that is not acceptable for real deployment. Applying QAT to the same model recovers accuracy to 91.45 percent. A loss reduced to 1.73 percentage points means QAT recovered 94.5 percent of the loss PTQ caused ($(31.27 - 1.73) / 31.27 = 0.945$). The remaining 1.73 percentage points is not negligible, but it is a qualitatively different risk class from PTQ’s 31.27 percentage points.

Looking at the QAT training curve explains why the observer and batch normalization freeze order matters. From fine tuning epoch 1 through epoch 10, validation accuracy oscillated wildly between 0.22 and 0.87, because observers were still updating activation scales and training had not stabilized. Oscillation continued even after observers were frozen at epoch 11, but once batch normalization running statistics were frozen at epoch 13, validation accuracy jumped to 0.90 and finished at 0.9111 by epoch 15. The sequential freeze of observers, then batch normalization, was the turning point for convergence. Skipping this order likely prevents this recovery, although this experiment did not test that directly; the oscillation pattern supports the hypothesis.

The extra cost of QAT is fine tuning time. At this model scale, 15 epochs took 90.8 seconds on CPU. That this is much slower per epoch than FP32 baseline training (80 epochs, 37.8 seconds) is due to fake quantization operator overhead, but the total fine tuning time itself is negligible at this model scale.

For reference, fp16 (casting the model to 16 bit with no retraining) was also measured in the same run. Its accuracy loss, 0.18 percentage points, was smaller than QAT’s. But its size reduction, 46.9 percent, is smaller

than INT8’s, and because this CPU environment has no native fp16 kernel, latency actually increased nearly fourfold, to 1.086 ms. The assumption that “16 bit means faster” did not hold on this hardware. fp16 is not the primary comparison of this paper, but it is left in as a counterexample showing that reducing precision does not automatically mean a speed gain.

6 Limitations

Before generalizing these results, the following boundaries should be stated clearly.

First, these results are specific to this particular AMC CNN and to this particular 6dB SNR condition. It cannot be assumed without verification that the same conclusion transfers to a larger, structurally different model, for example a device fingerprinting embedding model using attentive statistics pooling. Different architectures and different reliance on batch normalization could expose different PTQ failure modes.

Second, the QAT fine tuning learning rate and freeze schedule used here (learning rate $3e-5$, observer freeze at epoch 11, batch normalization freeze at epoch 13) was tried as a single configuration. A hyperparameter sweep would likely find a point with lower loss than 1.73 percentage points, but that sweep was not performed in this experiment.

Third, the large validation accuracy oscillation during the first 12 epochs of fine tuning itself suggests this schedule may not be optimal. Whether a gentler observer freeze schedule or a lower initial learning rate would reduce that oscillation was not checked.

Fourth, as the fp16 result shows, latency gains are strongly hardware dependent. The latency numbers in this paper were measured on one specific CPU with the qnnpack backend, and relative rankings could differ on other architectures, for example hardware with native ARM NEON fp16 support or a GPU.

Fifth, this experiment also measured dynamic quantization (which converts only Linear layers to INT8) on the same model, but it was not included in the main comparison. For reference, dynamic quantization reduced size by only 8.4 percent while latency increased by 18 percent. This is a negative result showing that dynamic quantization offers little value when most parameters live in convolutions and the fully connected layers are small.

7 Reproduction

This experiment can be reproduced with the following procedure.

Extract the 11 modulation classes from the 6dB SNR subset of RadioML2016.10a, keeping class balance, and split into train, validation, and test sets. The test set must not be used at any training stage.

The model is three Conv1d blocks (channels 2 to 32 to 64 to 64, each block including batch normalization and ReLU), followed by global average pooling and two fully connected layers (64 to 64 to 11).

Train the FP32 baseline with label smoothing of 0.05, early stopping after 20 epochs without validation improvement.

For PTQ: (1) fuse convolution, batch normalization, and ReLU; (2) insert observers and calibrate on the validation set; (3) convert to integer kernels with the qnnpack backend. There is no retraining step.

For QAT: (1) group convolution, batch normalization, and ReLU into a trainable fused module; (2) insert fake quantization observers on weights and activations; (3) fine tune for a few epochs at roughly one tenth the FP32 learning rate, freezing observers first and batch normalization running statistics second late in the schedule; (4) convert to integer kernels with the qnnpack backend.

For measurement: at batch size 1 on CPU, run 30 warmup passes followed by 300 measured forward passes and compute p50 and p95 latency from the timings; compute model size as the byte count of the serialized parameter dictionary. Every reported number should be a value the measurement code computes directly, never a value the model reports about itself.

8 References

1. B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” arXiv:1712.05877, 2017.
2. R. Krishnamoorthi, “Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper,” arXiv:1806.08342, 2018.
3. T. J. O’Shea, T. Roy, and T. C. Clancy, “Over-the-Air Deep Learning Based Radio Signal Classification,” IEEE Journal of Selected Topics in Signal Processing, vol. 12, no. 1, 2018 (source of the RadioML2016.10a dataset).
4. PyTorch Quantization documentation, “Quantization Recipe” and “(beta) Static Quantization with Eager Mode in PyTorch,” pytorch.org/docs.